Releasing

Releasing gemstone-js

`gemstone-js` should be released from a clean tag after the CI workflow passes on `main`.

Package Artifact

CI runs:

```
npm install --omit=optional
npm run verify
npm run release:check
npm pack --json
node scripts/write-checksums.mjs .tgz
node scripts/verify-checksums.mjs SHA256SUMS.txt
```

The workflow uploads the npm tarball as a GitHub Actions artifact so the exact package contents can be inspected before publishing. CI also uploads `SHA256SUMS.txt` for the generated tarball after verifying it. The `npm run verify` step includes `npm run examples:check` and `npm run public-surface:check` so packaged examples, the example catalog, and export-barrel changes are checked before packaging, and `npm run api-contract` so the runtime package entrypoint, package metadata, schema exports, and CLI bin targets are compared with the committed API contract. `npm run release:check` builds the same disposable npm tarball flow used for release inspection, writes `SHA256SUMS.txt`, verifies it, and confirms the tarball contains the expected source, schema, and release helper files. It also checks that every `main`, `types`, `exports`, and CLI `bin` target in `package.json` resolves to a file inside the tarball. `npm run provenance:check` runs the offline self-test for the saved npm provenance metadata validator used after publishing.

Artifact Inspection

Before `npm publish`, compare the CI tarball with a local package:

```
npm pack --json
node scripts/write-checksums.mjs .tgz
node scripts/verify-checksums.mjs SHA256SUMS.txt
npm run release:check
npm run provenance:check
npm run native-install:check
npm run release-candidate:check -- --skip-live
npm run ci-artifact:review -- --dir .
tar -tzf gemstone-js-*.tgz
shasum -a 256 -c SHA256SUMS.txt
npm run api-contract:installed
```

Check that the tarball contains compiled `dist/`, source `src/`, `docs`, `schemas`, `examples`, the codegen scripts, API contract scripts, `README.md`, `LICENSE`, and `package.json`. It must not contain `tests/`, `tsconfig*.json`, optional native binaries, local caches, or editor files. Verify that both checked-in generated files are present: `examples/codegen.generated.ts` and `examples/booking.decorators.generated.ts`, that the web adapter examples are present, and that the public API contract file `scripts/public-surface.expected.json` is present. The `scripts/check-release-artifacts.mjs` check performs this package-content and checksum

validation in a temporary directory without leaving tarballs in the working tree. The extracted-artifact check also verifies that published CLI bin targets exist and keep their Node shebangs, and that each installed CLI target can render `--help` from the published layout. It also verifies that the installed example catalog includes the web and quickstart examples. It runs the packaged `scripts/check-release-artifacts.mjs` and `scripts/verify-provenance-metadata.mjs --self-test` helpers from the extracted tarball so release verification scripts are tested in their published layout. For a native-enabled beta, `npm run native-install:check` packs both this project and the sibling `../gemstone-js-native` checkout, installs both tarballs into a disposable project, confirms that `@gemstone-js/native` exposes `createGciSessionWorker()`, verifies that `gemstone-js` selects the `node-worker` runtime, checks npm pack integrity/shasum metadata, confirms `gemstone-js` declares the exact sibling `@gemstone-js/native` version in `optionalDependencies`, inspects the native tarball for worker files and a `.node` binary, and validates the installed dependency graph with `npm ls`. Then run a live smoke with `GS_RUN_LIVE=1 GS_NATIVE_SESSION_WORKER=1` against the installed packages. For the final beta release-candidate pass, run `GS_RUN_LIVE=1 npm run release-candidate:check`; it performs the local verify gate, installed JS/native package proof with worker-mode live smoke, and the checkout worker live regression.

To review the exact GitHub Actions package artifact before publishing:

```
gh run list --workflow CI --branch main --limit 5
gh run download <run-id> --name gemstone-js-package-node-24 --dir /tmp/gemstone-js-ci-artifact
npm run ci-artifact:review -- --dir /tmp/gemstone-js-ci-artifact/gemstone-js-package-node-24
```

The review command verifies `SHA256SUMS.txt`, checks the tarball filename and version against the local `package.json`, confirms provenance and native optional-dependency metadata, inspects required package files, rejects test, cache, Git, and workflow directories, and checks package entrypoint/bin targets.

Provenance Verification

After publishing, verify registry metadata and signatures from a disposable project:

```
VERSION=$(node -p "require('./package.json').version")
npm view gemstone-js@$VERSION dist.integrity dist.signatures --json > npm-provenance.json
node scripts/verify-provenance-metadata.mjs npm-provenance.json
npm install gemstone-js@$VERSION
npm audit signatures
```

The metadata must include `dist.integrity` and `dist.signatures`, and `npm audit signatures` must complete without signature or provenance failures. `npm run provenance:check` covers the offline validator with fixture metadata, so a malformed saved registry response is rejected before manual inspection.

Publish Checklist

1. Verify `package.json` has the expected version, repository, license, and `publishConfig.provenance`.
2. Run `npm run verify` locally. This includes `npm run release:check`.
3. Run `npm run public-surface:check` if the public barrel changed, and review any intentional export changes before regenerating the contract with `npm run public-surface:write`.
4. Run `gemstone-js-api-contract --json` against the packed or installed package if the package entrypoint changed.
5. Review the CI tarball artifact contents and checksum file with the artifact inspection checklist above, or run `npm run ci-artifact:review -- --dir <downloaded-artifact-dir>`.
6. Run `GS_RUN_LIVE=1 npm run release-candidate:check` against the candidate native package artifacts.
7. Publish with provenance using a prerelease dist-tag:

```
npm publish --access public --tag alpha --provenance
```